

PRÁCTICO UNIDAD 3
PROGRAMACIÓN ORIENTADA A OBJETOS

Requerimientos generales

La resolución de cada ejercicio debe:

- Usar clases independientes.
- Incluir manejo de excepciones

Ejercicio Nº 1: Composición

El **sistema de administración** de las clínicas privadas **"San Juan"** y **"Santa Lucia"** requiere modelar la relación entre la clínica y sus consultorios médicos. En este contexto, cada Clínica está compuesta por múltiples consultorios, y cada consultorio pertenece exclusivamente a una única Clínica. El objetivo de este modelo es permitir una gestión eficiente de los espacios de atención médica, garantizando que cada consultorio esté correctamente asociado a la clínica para administrar: disponibilidad, asignación a profesionales, mantenimiento, y estado de ocupación.

Se necesita conocer:

De cada Consultorio: identificador de la clínica, número de consultorio, piso, especialidad asignada, costo de uso diario, y estado (Libre / Ocupado / Mantenimiento).

De la Clínica: identificador de la clínica, Nombre, Dirección, Director, una lista de los consultorios que posee y cantidad de pisos.

Se pide:

- 1 Definir las clases **Clínica** y **Consultorio** con sus atributos privados y los métodos necesarios.
- 2 Definir una clase **Gestor Clínica**, que permita instanciar los objetos Clínica leyendo los datos desde el archivo: **"clínicas.csv"**, cargar los consultorios correspondientes a cada clínica que permita instanciar los objetos Consultorio desde el archivo **"consultorios.csv"**, este archivo se encuentra ordenado por id_ de la clínica. Ambos archivos poseen el separador ";".
- 3 Implementar un menú de opciones que permita:
 - a. Agregar consultorios a la clínica: Permitir ingresar por teclado los datos de un nuevo Consultorio y asociarlo a la clínica correspondiente.
 - b. Asignar un consultorio a un profesional médico: Dado el número de consultorio, registrar dicho consultorio como **ocupado**.
 - c. Liberar un consultorio: Dado el número de consultorio, registrar nuevamente el estado a **libre**.
 - d. Dada una especialidad médica: (Cardiología, Pediatría, Traumatología, Clínica Médica, etc.), mostrar: número de consultorio, piso, estado actual, de todos los consultorios que correspondan a dicha especialidad, ordenado de menor a mayor por piso y número de consultorio. Para realizar este ítem sobrecargar el operador `__lt__`, en la clase Consultorio.
 - e. Leer por teclado el nombre de una clínica, y mostrar la cantidad de consultorios libres por piso.
 - f. Para cada especialidad mostrar el detalle asociado con el siguiente formato:

Especialidad: xxxxxxxxxxxx

Número	Piso	Costo diario	Estado
xxxxxx	xx	xxxx.xx	xxxxx
xxxxxx	xx	xxxx.xx	xxxxx

Ejercicio Nº 2: Agregación

Contexto

Una plataforma de organización de eventos llamada "EventosYA" gestiona los eventos y sus invitados. Cada evento puede tener múltiples invitados, pero los invitados existen en el sistema independientemente del evento (pueden asistir a más de uno).

Se necesita conocer:

De los invitados: nombre, DNI, email y dirección.

De los eventos: nombre del evento, fecha y tipo (conferencia, cumpleaños, boda, etc.).

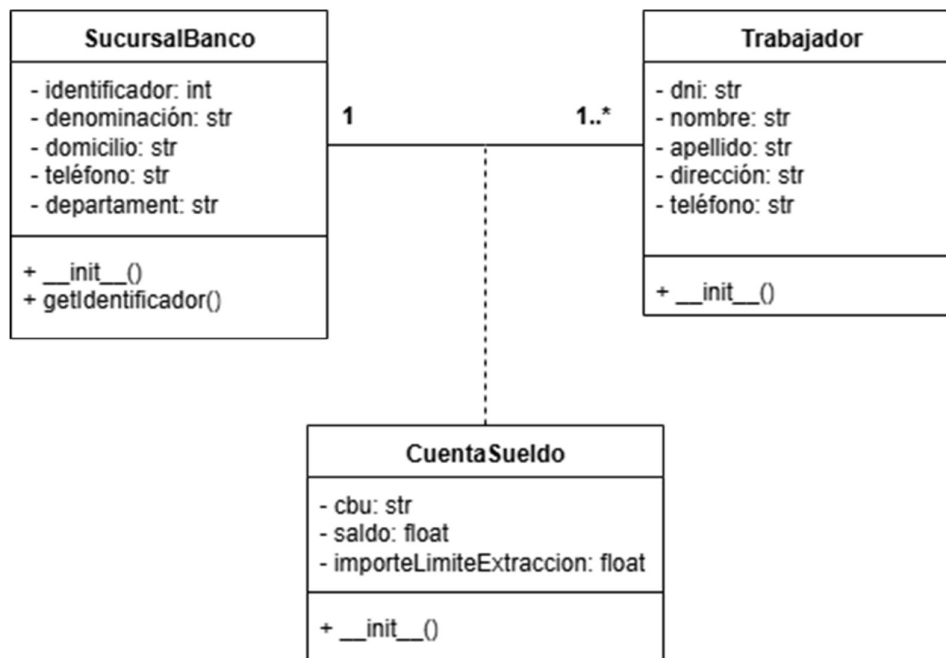
Se pide:

- 1- Definir las clases Invitado y Evento con sus atributos y métodos necesarios.
- 2- Crear una clase Gestor de Eventos implementando una lista Python que permita instanciar los objetos.
- 3- Implementar un menú de opciones que permita:
 - a. Crear y agregar eventos al gestor.
 - b. Crear y agregar invitados a un evento.
 - c. Dado un tipo de evento ingresado por teclado, mostrar los nombres de los invitados que asisten a ese tipo de evento.
 - d. Dado un DNI, mostrar si el invitado está registrado en algún evento: si está, mostrar nombre(s) del/de los evento(s); si no, informar que no está registrado.
 - e. Dada una fecha ingresada por teclado, listar los eventos de tipo "conferencia" programados para esa fecha.

Ejercicio Nº 3: Clase Asociación

Contexto:

El Banco "SJExpress SRL", posee varias sucursales en la provincia de San Juan, en la que Trabajadores de la provincia poseen Cuentas Sueldo. Luego del análisis del contexto el Analista, le solicita que implemente la siguiente relación:



Se pide:

- 1- Definir las clases SucursalBanco, Trabajador y CuentaSueldo con sus atributos privados y métodos necesarios.
- 2- Genere un Gestor de Sucursales, implementado en una lista Python. Los datos de las Sucursales vienen en un archivo separado por “;”, denominado “**sucursales.csv**”.
- 3- Genere un Gestor de Cuentas Sueldo, implementado en una lista definida por el programador. Los datos de las Cuentas Sueldo están almacenados en un archivo separado por “;”, denominado “**cuentasSueldo.csv**”. Los datos de los Trabajadores están almacenados en un archivo separado por “;”, denominado “**trabajadores.csv**”.
- 4- Implementar un menú de opciones que permita:
 - a. Leer por teclado el identificador de una sucursal, y listar dni, nombre, apellido de los trabajadores que tienen Cuenta Sueldo en dicha sucursal, listando también: cbu y saldo de su Cuenta Sueldo.
 - b. Leer por teclado el cbu de una Cuenta Sueldo, y un importe a extraer, informar saldo luego de la extracción, dni, nombre y apellido del Trabajador al que corresponde dicha Cuenta Sueldo.
 - c. Leer por teclado el identificador de una Sucursal, y el dni de un trabajador, e informar: nombre y apellido del trabajador, cbu de la cuenta bancaria y saldo de la misma.
 - d. Leer por teclado el dni de un trabajador y el importe a depositar en su Cuenta Sueldo, actualizar el saldo de la misma, mostrando cbu y nuevo saldo.

NOTA 1: complete las clases del sistema con los métodos necesarios para dar respuesta a las funcionalidades anteriores.

NOTA 2: el analista del sistema, le solicita que aplique test de unidad, utilizando **unittest**, para verificar el correcto funcionamiento de la clase Cuenta Sueldo, y de la lista definida por el programador. Si es necesario, actualice los métodos en las clases intervinientes en los test.

PARA LOS SIGUIENTES EJERCICIOS MAXIMICE LA REUSABILIDAD EN LOS MÉTODOS, SIEMPRE QUE ENCUENTRE UNA O MÁS SENTENCIAS REPETIDAS, DEBERÁ GENERALIZAR LO QUE SE REPITA EN LA CLASE BASE.

Ejercicio Nº4: Herencia y métodos con ligadura dinámica (Polimorfismo)

Contexto

Una empresa de turismo ofrece distintos tipos de excursiones para sus clientes. Las excursiones disponibles pueden ser: excursiones terrestres, y excursiones náuticas.

De todas las excursiones se necesita conocer: nombre de la excursión, destino, precio base.

De una **excursión terrestre** se conoce, además: cantidad de kilómetros del recorrido, si incluye almuerzo (True/False).

De una **excursión náutica** se conoce, además: duración en horas, cantidad máxima de pasajeros. El importe final que debe pagar el turista por cada excursión se calcula en función del precio base y de sus características, teniendo en cuenta las siguientes reglas de negocio:

Importe final de una excursión terrestre

Se calcula como: precio base + \$500 por cada 100 km de recorrido, y si incluye almuerzo se adiciona un 8% sobre el precio base.

Importe final de una excursión náutica

Se calcula como: precio base + un 15% por gastos de mantenimiento de embarcación.

El analista de la empresa le solicita a usted desarrollar una aplicación con las siguientes restricciones:

- 1 Definir la jerarquía de clases con los atributos y métodos correspondientes a la narrativa dada.
- 2 Almacenar en un Gestor de Excursiones tipo **lista definida por el programador** todas las excursiones que brinda la empresa. La lista deberá proveer los métodos y atributos necesarios para que sea un iterable.
- 3 Implementar un programa principal con un menú de opciones que permita testear las siguientes acciones:
 - a. Agregar excursiones a la colección.
 - b. Dada una posición de la lista, si la posición es incorrecta, lanzar la excepción `IndexError`, si la posición es correcta, mostrar por pantalla qué tipo de excursión se encuentra almacenada en dicha posición (usar `isinstance()`).
 - c. Mostrar la cantidad de excursiones de cada tipo.
 - d. Recorrer la colección y mostrar para todas las excursiones: nombre de la excursión, destino e importe final a pagar.

Nota: la excepción `IndexError`, debe lanzarse en el método de la lista, y el manejo de la misma, debe hacerse en menú/programa principal.

Ejercicio Nº5: Interfaces - Excepciones

Defina una interface con los siguientes métodos:

- a- **Insertar Elemento:** para insertar un objeto en una posición determinada en una colección, teniendo en cuenta el manejo de excepciones (`IndexError`) cuando la posición donde se vaya a insertar no sea válida.
- b- **Agregar Elemento:** para agregar un elemento al final de una colección.
- c- **Mostrar Elemento:** dada una posición de la colección, mostrar los datos del elemento almacenado en dicha posición si esa posición es válida, en caso de que no sea válida lanzar una excepción que controle el error.

Ejercicio Nº 6: Herencia y Archivos JSON

Contexto

La Organización “Innovar Ciencia” necesita desarrollar un sistema para administrar distintos eventos científicos realizados en diferentes universidades del país. Los eventos científicos pueden ser de 3 tipos: Congresos, Seminarios y Talleres.

Se necesita conocer:

- De todos los **eventos científicos**: código del evento, título, institución organizadora, fecha de realización, cantidad máxima de asistentes.
- De los **congresos** se necesita conocer, además: cantidad de conferencias principales, una lista de disertantes invitados.
- De cada **disertante** se conoce: nombre, especialidad, universidad de procedencia
- Para los **seminarios** se necesita conocer, además: temática principal, duración en horas, responsable académico.

- Para los **talleres** se necesita conocer, además: cantidad de actividades prácticas, nombre del coordinador, una lista de materiales requeridos

- De cada **material requerido** se conoce: nombre, cantidad, costo estimado

Nota: tener en cuenta que hay clases que incluyen a otras clases.

Se pide

1. Definir la jerarquía de clases correspondiente al problema planteado.
2. Implementar una colección utilizando una lista enlazada definida por el programador, cuyos nodos almacenen objetos pertenecientes a la jerarquía de clases de eventos científicos. La lista deberá proveer los métodos y atributos necesarios para que sea un iterable, e implementar la **interface del ejercicio 5**
3. Leer y procesar el archivo “eventos.json” para almacenar los eventos en la colección.
4. Implementar un programa principal con un menú de opciones que permita realizar las siguientes operaciones:
 - a. Insertar un evento científico en una posición determinada de la colección. El método insertar deberá lanzar la excepción `IndexError` si la posición ingresada no es válida. El programa principal deberá controlar dicha excepción.
 - b. Agregar un evento científico al final de la colección. El método agregar deberá validar que el objeto pertenezca a la jerarquía de clases correspondiente. En caso contrario deberá lanzar la excepción `TypeError`.
 - c. Mostrar todos los eventos almacenados indicando: tipo de evento, código, título, fecha, institución organizadora
 - d. Dado el título de un congreso, mostrar: cantidad de conferencias principales, listado de disertantes invitados, e universidad de procedencia de cada uno.
 - e. Dado el nombre de un coordinador, listar todos los talleres que coordina indicando: código del taller, Título, cantidad de actividades prácticas y la lista de materiales requeridos.
 - f. Actualizar el archivo “eventos.json” con las modificaciones realizadas en la colección.

Ejercicio Nº 7: Herencia Múltiple – Polimorfismo

En el ámbito universitario los agentes que ahí trabajan se pueden clasificar de la siguiente manera: Docentes, investigadores, docente investigador y personal de apoyo.

La información del personal universitario se encuentra almacenada en un archivo denominado “personal.json”.

El Centro de cómputos de la UNSJ, lo contrata a usted como programador, para que desarrolle una aplicación que permita procesar el archivo “.json” donde guarda la información del personal que trabaja en la misma. Para ello le provee la siguiente narrativa, que ha sido elaborada por el analista funcional:

-De todo el **personal** se conoce la siguiente información: cuil, apellido, nombre, sueldo básico y antigüedad.

Si es un **docente** se registra además carrera en la que dicta clases, cargo y cátedra.

Si es **personal de apoyo** se registra además categoría.

Si es un **investigador** se registra además área de investigación y tipo de investigación.

Si es un **docente investigador** se registra además carrera en la que dicta clases, cargo, cátedra, área de investigación y tipo de investigación, categoría en el programa de incentivos de investigación, importe extra por docencia e investigación.

Para cumplir con las tareas solicitadas por el analista, usted deberá:

- a) Definir la jerarquía de clases correspondiente a la narrativa dada.
- b) Almacenar en una colección tipo Lista definida por el programador, los agentes de la Universidad, obteniendo los datos del archivo "personal.json", la misma deberá implementar la interfaz definida en el **Ejercicio 5**. Los nodos de la lista, serán referencias a objetos que representen objetos de la clase base de la jerarquía.
- c) Implementar un programa principal con un menú de opciones que permita testear las siguientes acciones:

1. Insertar a agentes a la colección.
2. Agregar agentes a la colección.
3. Dada una posición de la lista: Mostrar por pantalla que tipo de agente se encuentra almacenado en dicha posición.
4. Ingresar por teclado el nombre de una carrera y generar un listado ordenado por nombre con todos los datos de los agentes que se desempeñan como docentes investigadores.
5. Dada un área de investigación, contar la cantidad de agentes que son docente investigador, y la cantidad de investigadores que trabajen en esa área.
6. Recorrer la colección y generar un listado que muestre nombre y apellido, tipo de Agente y sueldo de todos los agentes, ordenado por apellido.
7. Dada una categoría de investigación (I, II, III, IV o V), leída desde teclado, listar apellido, nombre e importe extra por docencia e investigación, de todos los docentes investigadores que poseen esa categoría, al final del listado deberá mostrar el total de dinero que la Secretaría de Investigación debe solicitar al Ministerio en concepto de importe extra que cobran los docentes investigadores de la categoría solicitada.
8. Almacenar los datos de todos los agentes en el archivo "personal.json"

Reglas de negocio para el cálculo del sueldo:

Por cada año de antigüedad el sueldo se incrementa en un porcentaje sobre el sueldo básico, por ejemplo: si tienen 5 años de antigüedad el sueldo sería sueldo básico + 5%(básico).

Porcentaje por cargo: 10 % si el cargo es simple, 20% si el cargo es semiexclusivo, 50% si el cargo es exclusivo.

Porcentaje por categoría: 10% si la categoría es de 1 a 10, 20 % si la categoría es de 11 a 20, 30% si la categoría es de 21 a 22.

Todos los porcentajes se calculan sobre el sueldo básico.

$\text{sueldoPersonalDeApoyo} = \text{Sueldo Básico} + \% \text{antigüedad} + \% \text{por categorías}$

$\text{sueldoDocente} = \text{Sueldo Básico} + \% \text{antigüedad} + \% \text{por cargo}$

$\text{sueldoInvestigador} = \text{Sueldo básico} + \% \text{antigüedad}$

$\text{sueldoDocenteInvestigador} = \text{sueldoDocente}() + \text{importe extra por docencia e investigación.}$

Ejercicio 8

Usar la narrativa del Ejercicio N° 7

Se ha agregado nuevos requerimientos al sistema, y usted como programador, debe darles solución:

El sistema prevé que los agentes serán administrados a través de una colección. Existen dos interfaces distintas para operar con agentes, la interfaz del **Tesorero**, que solamente puede acceder a los gastos que la universidad tiene en concepto de sueldos, para ello, dado un número de documento, podrá consultar el gasto de sueldos para el agente al que pertenece dicho número de documento.

La interfaz del **Director**, permite modificar el sueldo básico de todos los agentes, el porcentaje que se paga por cargo a un docente, el porcentaje que se paga por categoría a un personal de apoyo, y el porcentaje extra que se paga a un docente investigador; para ello debe proveerse el número de documento del agente, y el valor que corresponda según lo que se quiera modificar.

Usted deberá crear las interfaces mencionadas con las funcionalidades solicitadas.

```
class ITesorero (Interface)
```

```
    def gastosSueldoPorEmpleado ( dni):  
        pass
```

```
class IDirector (Interface)
```

```
    def modificarBasico(dni, nuevoBasico):  
        pass  
  
    def modificarPorcentajeporcargo(dni, nuevoPorcentaje):  
        pass  
  
    def modificarPorcentajeporcategoría(dni, nuevoPorcentaje):  
        pass  
  
    def modificarImporteExtra(dni, nuevoImporteExtra):  
        pass
```

En el programa principal agregue una opción para el Tesorero y un menú de opciones para Director. Para ello deberá autenticar al usuario que quiere acceder, si es tesorero, accederá a las funcionalidades de tesorero y si es director, accederá a las funciones de director. La autenticación del usuario se hace pidiendo nombre de usuario y contraseña.

Usuario/contraseña para Tesorero: uTesorero/ag@74ck

Usuario/contraseña para Director: uDirector/ufC77#!1